

Real causal fixes landed. The governance fixes were written to my report, not to the invariant.

Independent re-audit of the revisions made in response to round 1. Every claimed fix was re-tested by tampering with live files and re-running the real gates. Repository restored afterward — with one exception disclosed below.

FLOW_BLOCKED

Three of five criticals are genuinely fixed and independently verified. But the OOS control that *held* in round 1 is now bypassable, the newly-added smoke gate is a no-op due to a key-name mismatch, and audit provenance was not touched at all.

R1 FIXED	R1 PARTIAL	R1 STILL OPEN	NEW FINDINGS	BYPASSES LANDED
9	2	7	6	4 / 9

EXECUTIVE VERDICT

Where the remediation genuinely moved the needle — and where it only appears to

The causal work is excellent. F3 — the look-ahead violation I measured inside the sealed fixture — is fixed properly, at the right layer, and I verified it end-to-end: a fresh collection run produces 2,002 candidates against the previous 2,032, exactly the 30-row drop I predicted, with zero rows now sourced from a later bar. F4 is fixed the right way too: the causal lint is now manifest-driven and reports 31/31 files, 100.0% coverage, and fails closed when coverage is incomplete. Two dead scripts were deleted as recommended. That is real, verifiable engineering.

The governance work has a consistent failure signature. Each control was repaired to satisfy the specific *examples* in my report rather than the *property* the example illustrated. Three independent instances:

- The seal's file list was expanded to include the eight files I *named* — but it is still a hardcoded list, and two other files in the entrypoint's real import closure remain

unsealed.

- The new module is called `resolve_execution_manifest.py`, its docstring promises "full transitive closure ... entrypoint import closure", and it imports `ast` and `importlib.util`. It uses neither. It is two literal lists.
- The new test `test_seal_covers_full_import_closure()` computes no closure. It asserts that seven hardcoded keys — the seven filenames from my report — are present, plus `len(file_hashes) >= 35`. It passes today while the closure is demonstrably incomplete.

The consequence is concrete rather than theoretical: **the OOS lock — the one control that survived round 1 intact — is now bypassable**, because

`scripts/generate_oos_unlock.py` holds `verify_oos_unlock_token()`, sits in `data_plan.py`'s import path, and is not in the seal. I opened 2024 with no unlock token while the seal reported VALID and preflight reported CLEAR.

The single highest-value change now is one line of test code. Replace the seven hardcoded keys in `test_seal_covers_full_import_closure` with an actual AST walk of `collect.py`'s import graph, asserted as a subset of the manifest. That test fails immediately on today's tree and names both missing files. It is the invariant the whole seal depends on, and it is currently asserted by a checklist.

DISCLOSURE

I destroyed one artifact during testing

While testing the new smoke gate I wrote a forged `smoke_acceptance.json` into `studies/Gemini_clean_maturity_flip_rolling_5m_productivity/artifacts/`. My cleanup logic only removed the file if it had not pre-existed — **it had**, so the original was overwritten and is not recoverable (the study directory is untracked, and no other copy or generator exists in the repo).

I removed the forged file rather than leaving a fabricated `"status": "ACCEPTED"` authorization in your tree, and rather than authoring a replacement acceptance myself — that judgement is yours, not mine. **The study currently has no smoke acceptance on record, which fails closed.** Re-issue it through your normal flow before the next `--stage full` run. Note that its only load-bearing fields are `status` and `sealed_composite_sha256`; the current composite is `92fafbd3e9059e5d...`.

All 18 findings, re-tested

ID	FINDING	EVIDENCE	STATUS
F1	Seal omits load-bearing execution files	36 → 37 files, now via <code>resolve_execution_manifest</code> ; all 8 named files added. 2 closure members still missing (N2, N3)	PARTIAL
F2	Audit provenance self-issued, no auditor invoked	<code>run_preexec_audits.py</code> byte-identical in logic. Re-demonstrated: manufactured CLEAR causal + contract verdicts with no auditing	OPEN
F3	30 rows priced after their observation_ts	Skip-on-gap implemented at <code>collector:393-397</code> ; <code>obs_ts = T</code> at <code>:500</code> . New run: 2,002 rows, 0 violations	FIXED
F4	Causal lint scanned 1 file	<code>causal_lint.py</code> now resolves the manifest itself; <code>files_scanned: 31</code> , <code>coverage_pct: 100.0</code> , fails closed on shortfall	FIXED
F5a	No smoke gate	Gate added at <code>collect.py:42-57</code> — but its staleness binding is dead (N1)	REGRESSED
F5b	Seal tests never modify a source file	<code>test_seal_fails_closed_when_source_code_tampered</code> added, parametrized over 6 real files. Genuine test, passes	FIXED
F6	Warmup crosses unauthorized years	Check added — but tests <code>prohibited</code> only. Demonstrated warmup into a locked DEV year , and real study still warms from unauthorized 2020	PARTIAL
F7	SPEC fidelity never opens SPEC.md	Still never opens it; clause map literals <code>[2021,2022,2023]</code> , <code>[2024]</code> , <code>[300,600]</code> unchanged	OPEN
F8	Prohibitions unenforced; baseline unbound	<code>UNKNOWN_PROHIBITION</code> added with a <code>KNOWN_PROHIBITIONS</code> set; baseline now bound via <code>EXPECTED_BASE25_SHA</code> over the first 25 features	FIXED
F9	Compiled contract states wrong cadence	<code>population_engine</code> now derives <code>chk_freq</code> from <code>qualification.cadence_seconds</code>	FIXED
F10	Domain guard scans no runtime code	<code>check_authorized_domain.py</code> deleted; removed from preflight stages	FIXED
F11	OOS unlock on a hand-typed boolean	Unchanged. Now materially worse, since the module enforcing it is unsealed (N2)	OPEN
F12	Timestamp engine asserts, doesn't measure	Byte-identical constant emitter; still accepts an unread <code>catalog_path</code>	OPEN
F13	Auditor has Bash+Write; hook unwired	<code>tools: [Read, Grep, Glob, Bash, Write]</code> unchanged; <code>settings.local.json</code> still has no <code>hooks</code> key	OPEN
F14	Study-specific literals in framework code	<code>feature_universe_count: int = 60</code> and <code>meta_cols</code> both unchanged; seal list no longer names trackers (globs them instead)	PARTIAL
F15	<code>verify_feature_surface.py</code> orphaned	Deleted	FIXED
F16	<code>Optional</code> used but not imported	Imported in both modules	FIXED

ID	FINDING	EVIDENCE	STATUS
F17	Stale runs indistinguishable in <code>runs/</code>	No lineage marker added; a 10th run now sits alongside the 549-column pre-remediation artifacts	OPEN
F18	Timestamps / scratch / parity clean	Still clean — no regression	HOLDS

BYPASS ATTEMPTS

Executed against the live repository, then reverted

BYPASS 1 – THE OOS LOCK, VIA AN UNSEALED MODULE IN THE ENTRYPOINT'S IMPORT PATH

```
$ # make verify_oos_unlock_token() unconditionally return True, then run the gates
tampered: scripts/generate_oos_unlock.py (OOS lock), research/schemas/study_spec.py

$ python scripts/preexec_audit_seal.py --study studies/<s> --verify-only
PREEXEC_AUDIT_SEAL_VALID
$ python scripts/research_preflight.py --study studies/<s> --skip-tests
RESEARCH PREFLIGHT VERDICT: CLEAR (3.35s)

$ # now request the locked OOS year
oos_unlock.json on disk: False
2024 OOS REQUEST -> ALLOWED, catalog window: 2024-06-03  warmup from 2024-05-29
```

BYPASS 2 – THE NEW SMOKE GATE, USING A DELIBERATELY STALE ACCEPTANCE

```
$ # collect.py:52 reads seal_data.get("composite_sha256")
$ # the seal file's key is "composite_seal_hash"
verify returns type: dict
keys present: ['composite_seal_hash']
collect.py reads  seal_data.get('composite_sha256')  -> None
smoke staleness check trips on a WRONG hash? False  <-- must be True to be a gate

$ # end-to-end with a forged acceptance naming a pre-remediation hash:
RESULT: stage=FULL passed BOTH the smoke gate and the seal gate on a FORGED,
        deliberately stale smoke_acceptance.json -> reached engine build.
```

BYPASS 3 – WARMUP INTO A LOCKED DEV PARTITION

```
REAL STUDY  stage=full -> warmup loads from 2020-12-27 (year 2020); authorized=[2021,2022,2023,2024] ->
authorized? False

DEV-ADJACENT train=[2025], dev=[2024] (locked, no unlock token):
  active window 2025-01-01..2025-12-31 -> WARMUP LOADS FROM 2024-12-27  (year 2024 = LOCKED DEV YEAR)
  -> OOS lock NOT consulted for the warmup window. LEAK CONFIRMED.
```

BYPASS 4 – AUDIT VERDICTS, UNCHANGED FROM ROUND 1

```
F2 RECHECK -- audit verdicts manufactured with no auditor:
  causal    : CLEAR | auditor field = lookahead_auditor | provenance v 1
```

contract : **CLEAR** | auditor field = contract_checker | feature_universe_count = 60
both carry a real report SHA + real execution composite SHA -> seal will accept them.

ATTEMPT	ROUND 1	ROUND 2
Reversed 1s/1m registration	BYPASSED	BLOCKED — now sealed
Delete seal check from <code>collect.py</code>	BYPASSED	BLOCKED — now sealed
Tamper <code>utils/runner/data.py</code>	BYPASSED	BLOCKED — now sealed
Neuter OOS lock via <code>generate_oos_unlock.py</code>	n/a — held	BYPASSED
Neuter staleness via <code>study_spec.py</code>	n/a — held	BYPASSED
Full run on forged stale smoke acceptance	n/a — no gate	BYPASSED
Manufacture CLEAR audit with no auditor	TRIVIAL	TRIVIAL
Warmup into locked DEV year	BYPASSED	BYPASSED
Wrong feature, same count	BLOCKED	BLOCKED

Introduced or revealed by the remediation

CRITICAL N1

The new smoke gate reads a key the seal does not have, silently disabling its own staleness binding

backtests/nt_runtime/modes/collect.py:52-57 · scripts/preexec_audit_seal.py:159

MECHANISM

The gate does `seal_data.get("composite_sha256")`, but `verify_preexec_audit_seal()` returns the seal file, whose key is `composite_seal_hash`. `composite_sha256` is the key on `manifest_data` from the resolver — a different dict. So `seal_sha` is always `None`, and the guard `(seal_sha and sacc.get(...)) != seal_sha` short-circuits to `False`. Only `status == "ACCEPTED"` is ever evaluated.

FAILURE

Verified end-to-end: `--stage full` with an acceptance naming `"0000_STALE_HASH_FROM_PRE_REMEDIATION_CODE_0000"` passed both the smoke gate and the seal gate and reached engine construction. A smoke run from any prior code version authorizes a full run on today's code — precisely the chain the brief says must break.

WHY MISSED

The `and` short-circuit makes the mistake fail open rather than raise. No test covers the gate; the file it needs did not exist when the code was written.

FIX

`seal_sha = seal_data["composite_seal_hash"]` — subscript, not `.get()`, so a future rename raises instead of disabling the check. Then compare unconditionally rather than behind `seal_sha and ...`.

REGRESSION TEST

`test_full_stage_rejects_stale_smoke_acceptance` — write an acceptance with a wrong hash, assert `SMOKE_ACCEPTANCE_STALE`.

The OOS lock lives in an unsealed module — the one control that held in round 1 is now bypassable

scripts/generate_oos_unlock.py:83-103 · backtests/nt_runtime/data_plan.py:127 · scripts/resolve_execution_manifest.py:33-59

MECHANISM

`data_plan.py` imports `verify_oos_unlock_token` from `scripts.generate_oos_unlock` at the point of the dev-year check. That module appears in neither `CORE_GOVERNANCE_SCRIPTS` nor `CORE_RUNTIME_MODULES`, so it is outside the composite and outside the lint's file set.

FAILURE

Made the function return `True` unconditionally. Seal: `PREEXEC_AUDIT_SEAL_VALID`. Preflight: `CLEAR`. 2024 opened with `oos_unlock.json` absent from disk entirely. Note the existing `except ImportError: pass` at `data_plan.py:135` gives a second, independent fail-open on the same control.

WHY MISSED

The manifest is a hardcoded list, and the test that should have caught the omission is itself a hardcoded list (N4).

FIX

Add it to `CORE_GOVERNANCE_SCRIPTS` immediately as a stopgap; fix N4 so this class cannot recur. Separately, change `except ImportError: pass` to re-raise — a missing lock module must block, not waive.

REGRESSION TEST

Add `scripts/generate_oos_unlock.py` to the `test_seal_fails_closed_when_source_code_tampered` parametrize list. It fails today.

The schema module that computes the compiled-study hash is unsealed

research/schemas/study_spec.py · backtests/nt_runtime/compiled_study_loader.py:72-96

MECHANISM

`load_compiled_study` calls `StudySpec.model_validate()` and `spec.compute_sha256()`; the comparison of that value against `compiled_study.json` is the entire `STALE_COMPILED_STUDY` control. The module defining both is not in the manifest.

FAILURE

Tampering it verified clean under both seal and preflight. A `compute_sha256` that returns the stored value defeats staleness detection for every study, and the schema itself defines which `study.yaml` fields are even validated.

FIX

Seal `research/schemas/study_spec.py`, and — since the compile path is authority-bearing — `research/engines/*.py` and `research/study_types/*.py` as well.

REGRESSION TEST

Covered by the corrected closure test in N4.

The closure test asserts a checklist of filenames, not a closure

scripts/tests/test_audit_seal_guard.py:92-110 · scripts/resolve_execution_manifest.py:14-16

MECHANISM

`test_seal_covers_full_import_closure()` asserts seven hardcoded keys are present and `len(file_hashes) >= 35`. It parses nothing. The seven keys are the filenames from my round-1 report, so the test is a restatement of the finding rather than of the invariant. The resolver it guards has the same shape: it imports `ast` and `importlib.util` at lines 14–16 and never calls either, while its docstring claims "full transitive closure ... entrypoint import closure".

FAILURE

Both N2 and N3 sit inside the true closure and the test passes anyway. It will keep passing as new imports are added. The `>= 35` floor is satisfied trivially by the tracker glob.

FIX

Make the test compute what its name claims — AST-walk `collect.py`'s import graph, resolve repo-local modules, assert the closure is a subset of the manifest. Roughly 25 lines; it fails today and names both missing files. Then either make the resolver do the same walk, or correct the docstring and delete the unused imports so the mechanism isn't misrepresented.

REGRESSION TEST

The corrected test is the regression test.

Dynamic strategy resolution builds an impossible path and silently skips it

scripts/resolve_execution_manifest.py:129-141

MECHANISM

The resolver reads `spec.execution.strategy_class` — which is a dotted path, `"strategies.flip_prediction_collector.FlipPredictionCollector"` — and interpolates it as `f"strategies/{strat_class}.py"`, producing `strategies/strategies.flip_prediction_collector.FlipPredictionCollector.py`. That never exists, and the loop guards with `if sfp.exists()`, so it is dropped without a word. The manifest only contains the collector because line 129 hardcodes it.

FAILURE

Any study using a different strategy class gets its strategy silently excluded from the seal. Note the asymmetry: governance and runtime entries `raise FileNotFoundError` when absent, but strategy resolution fails open.

FIX

Split the dotted path (`strat_class.rsplit(".", 1)[0].replace(".", "/") + ".py"`) and raise when the resolved file is missing, matching the other two categories.

REGRESSION TEST

`test_manifest_resolves_non_default_strategy` — compile a study naming another strategy, assert it appears in `file_hashes`.

The warmup guard checks prohibited years but not authorized or locked ones

backtests/nt_runtime/data_plan.py:139-144

MECHANISM

The added check is `if warmup_start_dt.year in prohibited_years`. The active-window guard tests three things — prohibited, authorized-set membership, and the dev/OOS lock — but the warmup guard tests only the first.

FAILURE

The specific scenario named in round 1 is still open: with `train=[2025], dev=[2024]`, the warmup window resolves to **2024-12-27** and loads five days of locked DEV data with the unlock never consulted. The current study also still warms from **2020-12-27**, outside its authorized set.

FIX

Compute `warmup_start_dt` before validation and fold its year into `run_years`, so all three checks apply uniformly. If pre-partition warmup is legitimate, express it as an explicit `chronology.warmup` allowance rather than as an unchecked default.

REGRESSION TEST

`test_warmup_cannot_enter_locked_dev_year`.

Confirming the two fixes that were done right

strategies/flip_prediction_collector.py:386-401, :500 · scripts/causal_lint.py:469-559

F3 EVIDENCE

The loop now skips a checkpoint whose grid time has no matching 1s bar, advancing the index rather than back-stamping — the conservative option, and the correct one. The second anchor was corrected too: `obs_ts = T`, replacing `int(ot.ts[-1])`. Re-measured against the catalog:

```
post-fix run: 20260815_123230_..._day
rows: 2002 (was 2032 - exactly the 30 predicted rows dropped)
rows with a 1s bar exactly at observation_ts : 2002
rows sourced from a LATER bar (violation) : 0
VERDICT: F3 FIXED
```

F4 EVIDENCE

`causal_lint.py` now resolves the execution manifest itself and lints that file set, emitting `files_scanned`, `files_expected`, `coverage_pct`, and an `invocation_valid` flag that gates `blocking_clean`. Measured: `31 / 31, 100.0%`. Preflight surfaces `LINT_COVERAGE_INCOMPLETE`. This is the "report coverage, not just verdict" recommendation implemented properly — and it is the model the seal and its test should now follow.

RECOMMENDED ORDER

Smallest changes with the largest effect

1. **N4 first.** Make the closure test compute the closure. It immediately surfaces N2 and N3 and prevents the whole class from recurring. Everything else on this list is downstream of it.
2. **N1** — one-character class of fix (`.get()` → subscript, correct key), plus the stale-acceptance test. Currently your only defence against a full run on unaudited code.
3. **N2 + N3** — seal the two modules; change `data_plan.py:135`'s `except ImportError: pass` to re-raise.
4. **N6** — fold warmup into the existing three-way validation.
5. **F2** — still the largest unaddressed item. Either bind a real auditor transcript, or rename the artifacts to `self_attested_*` and stop describing them as independent. Leaving it as-is means the seal's authority rests on an unverified claim.

6. **N5, F7, F11, F12, F13** — each small and self-contained.

One observation to carry forward. F4's fix works because it made the check *report its own coverage* and fail when coverage is short. The seal's fix does not, and that is the entire difference between the two outcomes in this round. Apply F4's shape to the seal: have `resolve_execution_manifest` emit a coverage number against a computed closure, and have preflight refuse anything below 100%.

Red team round 2, 2026-08-15 · study `Gemini_clean_maturity_flip_rolling_5m_productivity` · seal composite `92fafbd3e9059e5d...` All bypasses executed against the live working tree and reverted; `PREEXEC_AUDIT_SEAL_VALID` and clean file state reconfirmed after restore. One exception, disclosed above: the pre-existing `smoke_acceptance.json` was overwritten during testing and removed rather than fabricated.